Member-only story

Karpathy's 4 CLAUDE.md Rules Cut Mistakes by 30%. I added 4 more to further cut it down to less than 5.

The 8-Rule Architecture That Fixes AI Agents

Shashwat Follow 5 min read · May 13, 2026

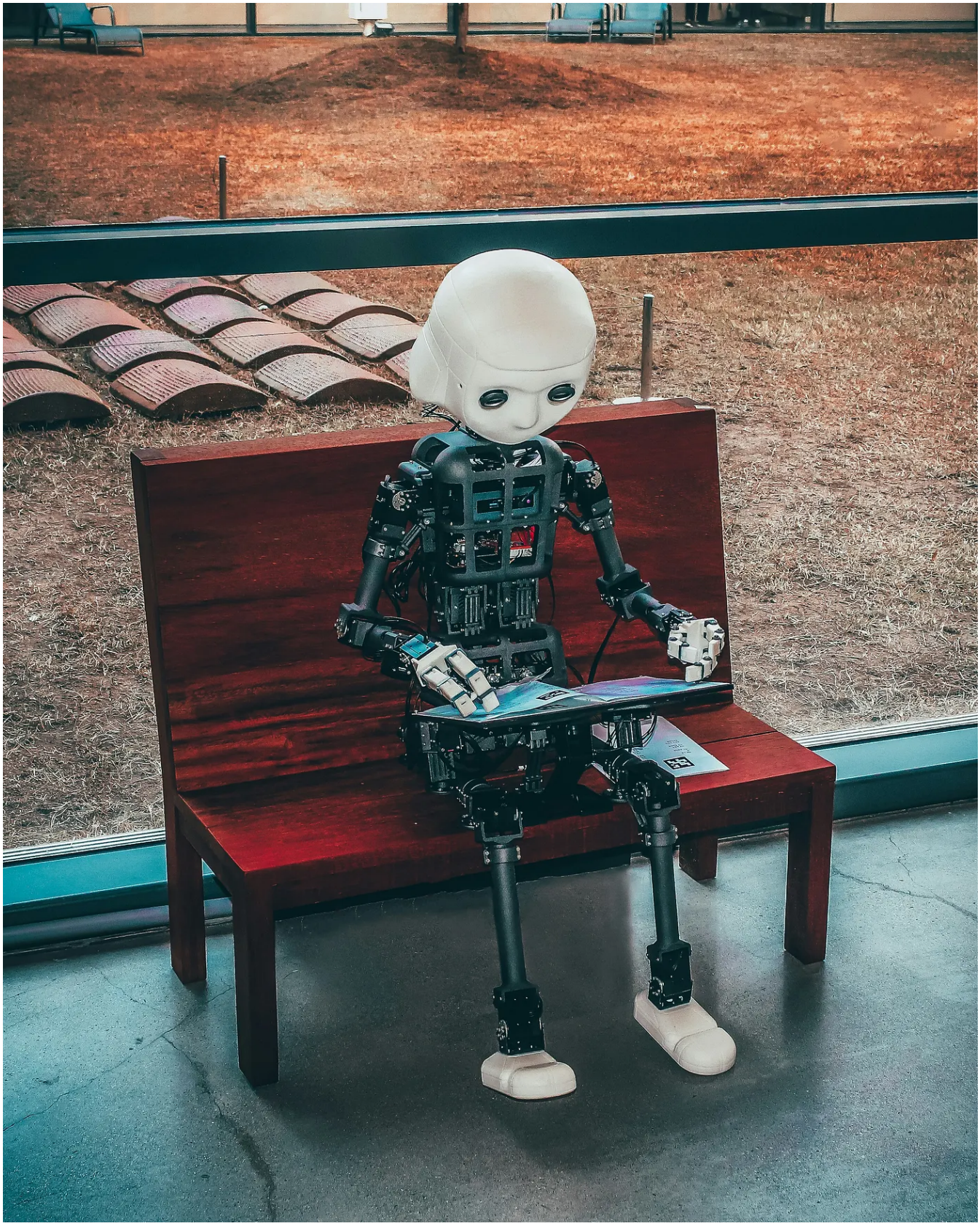

Photo by Andrea De Santis on Unsplash

Past 200 lines, prompt compliance plummets, and your agent silently ignores you xD

> *So, If you are not engineering your constraints, you are just wasting tokens!*

I'm sure you are aware that In January 2026, **Andrej Karpathy** publicly dissected why Claude Code was failing in production codebases:

silent assumptions, over-engineering, and orthogonal damage.

And,

A developer packaged those complaints into a streamlined, 4-rule `CLAUDE.md` file.

It exploded, becoming the fastest-growing GitHub repo of the year.

. . .

But now in May, the AI ecosystem moves even more aggressively.

Today, we are running more and more multi-step, autonomous agentic workflows.

> When I was deploying autonomous agents to refactor the data ingestion layer, I hit this friction:
>
> the agent would complete three steps perfectly, hallucinate on step four, and silently overwrite the working code.

Karpathy's original rules didn't protect against multi-step degradation.

. . .

free to read for non members

. . .

This proved that while the original four rules are the floor, they are not the ceiling.

By engineering exactly 4 *additional* rules specifically for modern agent orchestration, AI mistake rate drilled down further for me.

I took three from an operator on X and one from a blog!

. . .

The Foundation: Karpathy's 4 Rules

The viral implementation of Karpathy's complaints established the baseline. These four close roughly 40% of standard, single-prompt failure modes:

1. **Think Before Coding:** No silent assumptions. Push back if a simpler approach exists.
2. **Simplicity First:** Minimum code required. No speculative features.
3. **Surgical Changes:** Touch only what you must. Do not "improve" adjacent formatting.
4. **Goal-Driven Execution:** Define success criteria and loop until verified, rather than blindly following rigid steps.

Why They Break Today: These rules are entirely silent on multi-step pipelines. They don't give the agent a token budget, they don't force checkpoints, and they assume the agent already understands the surrounding codebase.

. . .

The Execution Layer: The 4 New Agentic Rules

5. Hard Token Budgets

Without budgets, a looping agent will burn a 50,000-token context dump debugging the same error message until it loses its mind.

The Rule: Establish a hard per-task budget (e.g., 4,000 tokens) and per-session budget (30,000 tokens). If breached, force the agent to summarize and restart the session. Surfacing a breach is always better than silently overrunning your API limits.

6. Read Before You Write

Karpathy's rules say "don't touch adjacent code," but they don't say "understand adjacent code." This leads to the AI blindly writing duplicate functions that already exist 30 lines away.

The Rule: Before adding code to a file, force the agent to read the file's exports, immediate callers, and shared utilities. "Looks orthogonal" is a dangerous assumption.

7. Checkpoint Multi-Step Operations

If a 6-step refactor goes wrong on step 4, the agent will happily execute steps 5 and 6 on top of the broken state, ruining the entire branch and forcing a manual `git reset`.

The Rule: After every significant step, the agent must summarize what

was verified and what is left. It cannot continue from a state it cannot describe back to you.

8. Fail Loud

The most expensive failures look exactly like successes. “Migration completed” is a lie if 30 database records were silently skipped due to constraint violations.

The Rule: Default to surfacing uncertainty. If the agent skipped anything, or if a test passed for the wrong reasons, it must fail loud and alert the user immediately.

. . .

Photo by Gabriel Heinzer on Unsplash

The reality is that past a certain length, Claude stops reading the rules and just pattern-matches the fact that "rules exist."

By restricting the file to exactly 8 high-impact, imperative rules, the compliance rate stays above 75%, and the error rate drops to near zero.

. . .

The Master `CLAUDE.md` Architecture

Copy this exact text, save it as `CLAUDE.md` in your repository root, and append any strictly necessary project stack details below it.

```
# CLAUDE.md — 8-Rule Architecture
These rules apply to every task in this project unless explicitly overrid
Bias: caution over speed on non-trivial work. Use judgment on trivial tas
```

```
## Rule 1 — Think Before Coding
State assumptions explicitly. If uncertain, ask rather than
guess.
Push back when a simpler approach exists. Stop when confused.

## Rule 2 — Simplicity First
Minimum code that solves the problem. Nothing speculative.
No features beyond what was asked. No abstractions for single-
use code.

## Rule 3 — Surgical Changes
Touch only what you must. Clean up only your own mess.
Don't "improve" adjacent code, comments, or formatting. Match
existing style.

## Rule 4 — Goal-Driven Execution
Define success criteria. Loop until verified.
Don't follow steps. Define success and iterate independently.

## Rule 5 — Token budgets are not advisory
Per-task: 4,000 tokens. Per-session: 30,000 tokens.
If approaching budget, summarize and start fresh. Surface the
breach.

## Rule 6 — Read before you write
Before adding code, read exports, immediate callers, shared
```

```
utilities.
If unsure why code is structured a certain way, ask.

## Rule 7 — Checkpoint after every significant step
Summarize what was done, what's verified, what's left.
Don't continue from a state you can't describe back. Stop and
restate.

## Rule 8 — Fail loud
"Completed" is wrong if anything was skipped silently.
"Tests pass" is wrong if any were skipped.
Default to surfacing uncertainty, not hiding it.
```

. . .

`CLAUDE.md` is a strict behavioral contract designed to close specific, costly failure modes.

Karpathy's initial complaints defined the problems of autocomplete coding.

We are now orchestrating autonomous, multi-step agents.

> Lock down your environments, enforce your token budgets, and demand checkpoints.

. . .

In case we are meeting for the first time, come over here, it'll be worth the roller coaster of articles that are gonna come up in the next few weeks.

I swear tracking these updates is a job in itself, lately.

Here's the list which I've built and keep adding on.

And If you need help for analyzing UFC fights, please check out BoutPredict :)

Andrej Karpathy | Skills | Artificial Intelligence | Claude Code | Software Engineering

Published in Tech and AI Guild

110 followers · Last published 3 days ago

Follow

Every new updates and thoughts about tech and AI

Written by Shashwat

691 followers · 15 following

Follow

Sharing everything I learn (mostly tech). Scaling boutpredict.aibucket.org to 150 paid & 10,000+ free monthly users.
Business Inquiry: send2shashwat@gmail.com